

Equity Selector

Research Pipeline and Strategy Validation Methodology

Project Documentation

21 August 2026

Contents

1	Executive summary	4
2	Research philosophy	4
2.1	Chronological split	4
2.2	Target-blind screening before target selection	5
2.3	Model accuracy and portfolio usefulness	5
2.4	Search efficiency without changing the test	5
3	End-to-end architecture	5
3.1	Script roles and hand-offs	7
4	Quantitative definitions	7
4.1	Feature and target reference	7
4.2	Feature families used in data creation	7
4.3	Target families used in data creation	8
5	Stage I - Data_Creation_Screening.py	8
5.1	Research universes	8
5.2	Source history and benchmark	9
5.3	Stock screening before feature engineering	9
5.4	Feature creation and target-blind feature screening	10
5.5	Target creation and chronological split	10
5.6	Target-specific feature screens	10
5.7	Stored research table	10
5.8	Intraday session conversion - Intraday Conversion.py	10
6	Stage II - Model_Fitting.py	11
6.1	Target-specific loading	11
6.2	Active model catalogue	11
6.3	Resume and configuration history	11
6.4	Chronological walk-forward folds	12
6.5	Progressive racing schedule	12
6.6	Finalists and full validation	12
6.7	Leaderboard metrics	12
6.8	Exact Testing Eligible gates	13
7	Stage III - Best_Model_Test.py	13
7.1	Test-eligible model selection	13
7.2	Final chronological fit/test	13
7.3	Stored final-test metrics	13
8	From final test to portfolio metadata	14
8.1	Final-test retention and Predictability Score	14

8.2	Fine-grained Portfolio Target Type	14
8.3	Horizon extraction	14
8.4	Cross-target Quality Score	14
9	Stage IV - Horizon Score Backtests.py	15
9.1	Research-period role	15
9.2	Predict once, vary horizon scores	15
9.3	Horizon-score catalogue	15
9.4	Fixed portfolio settings during horizon research	16
9.5	Benchmark-relative Backtest Quality	16
9.6	One-dimensional sensitivity freeze	16
9.7	Random-context screens	17
9.8	Exhaustive survivor search	17
10	Stage V - Backtest Database.py	17
10.1	Purpose	17
10.2	User-specified research windows	17
10.3	Three-year feature warm-up and one market download	17
10.4	Cached database schema	18
11	Stage VI - Backtest Simulations.py	18
11.1	Fine-grained portfolio signal structure	18
11.2	Universe scenarios	19
11.3	Portfolio optimizer	19
11.4	Parameter sweep	19
11.5	Position timing and P&L accounting	19
11.6	Stored experiment fields and summaries	20
12	Selecting simulations for precise testing	20
13	Stage VII - Precise Backtest.py	20
13.1	Exact chosen configuration	20
13.2	Daily diagnostic panel	21
13.3	Current diagnostic thresholds	22
13.4	Drawdown episodes and extreme days	22
13.5	Anomaly engine	22
13.6	Full risk summary	23
14	End of coded research	24
15	Storage architecture	24
15.1	Features_Targets_Data.db	24
15.2	Selected_Features.txt	24
15.3	Validation_Model_Fits/<STOCK_TYPE>.db	24
15.4	Final_Test_Results.db	24
15.5	Backtest_Features_Targets.db	24
15.6	Automated_Backtest_Results.db	25
15.7	Single_Backtest_Anomaly_Report.db	25
16	Intended research sequence	25
17	What counts as evidence at each layer	26
A	Key formula summary	26
A.1	Predictability Score	26
A.2	Quality Score	27
A.3	Backtest Quality used by horizon research	27
A.4	Horizon sensitivity ratio	27

A.5	Fine-grained target model weight and contribution	27
A.6	Combine targets inside one type	27
A.7	Apply target-type value	27
A.8	Overall stock score	28
A.9	Portfolio weights	28
A.10	Effective holdings	28
A.11	Rolling beta	28
A.12	Active return	28
B	Horizon-score configurations and broad portfolio sweep	28
C	Base universe definitions	29
C.1	High Liquidity 30	29
C.2	Medium Liquidity 30	29
C.3	Lower Liquidity 30	29
C.4	Sector-matched / Sector Spread 30	29
C.5	Liquidity Barbell 30	29
D	Current validation/search constants	30

1 Executive summary

Equity Selector is a multi-stage quantitative equity research system. Its end product is a portfolio allocation process, not a single price forecast. The pipeline uses screened market data, target-specific predictive models, model quality scores, horizon-aware signal weights, broad portfolio simulations and detailed backtests.

The research chain is implemented across eight main scripts: `Data_Creation_Screening.py`, `Intraday_Conversion.py`, `Model_Fitting.py`, `Best_Model_Test.py`, `Horizon Score Backtests.py`, `Backtest Database.py`, `Backtest Simulations.py` and `Precise Backtest.py`. The mathematical feature and target definitions are kept separately in the 40-page *Equity Selector - Features & Targets Quant Reference*, which contains 114 feature functions and 26 target functions. Each entry records its purpose, formula, required data, output columns, strengths and limits, and a short Python implementation.

The main daily research download in `Data_Creation_Screening.py` uses adjusted Yahoo Finance OHLCV data from 2013-01-01 to 2023-09-30, with $\hat{G}SPC$ over the same period as the market benchmark. The script supports 30-, 60- and 90-stock liquidity and composition universes. The supplied active example is the Liquidity Barbell 30 universe.

The project separates five questions:

1. **Data suitability:** are the stocks and feature columns suitable for research before target performance is considered?
2. **Target predictability:** which screened features can help each target, and which model/parameter sets remain competitive in walk-forward validation?
3. **Unseen model performance:** does the selected model still work on the final chronological model-test block?
4. **Portfolio usefulness:** how should model quality, target meaning, prediction horizon, target-type value, position limits, concentration and rebalance frequency turn predictions into portfolio weights?
5. **Strategy behaviour:** when a chosen portfolio performs unusually well or poorly, which holdings, contributions, exposures, drawdowns, concentration states, turnover events and liquidity/capacity conditions explain it?

The pipeline moves evidence through increasingly selective stores. It starts with `Features_Targets_Data.db` and `Selected_Features.txt`, moves model evidence through stock-type validation databases and `Final_Test_Results.db`, creates the strategy cache in `Backtest_Features_Targets.db`, writes broad simulations to `Automated_Backtest_Results.db`, and writes detailed diagnostics to `Single_Backtest_Anomaly_Report.db`.

2 Research philosophy

2.1 Chronological split

The project uses chronological partitions throughout. Each part of the data has one job:

- earlier observations fit models,
- later development observations validate model choices,
- a later block is used for the final model test,
- post-development observations are used for strategy validation,
- a final later block remains untouched for the final strategy test.

Walk-forward validation and target purging keep the same ordering inside model development.

2.2 Target-blind screening before target selection

The project first checks whether a stock or feature is structurally suitable without using target performance. Only after that stage are features tested for possible usefulness to individual targets. This keeps basic data-quality decisions separate from target results.

2.3 Model accuracy and portfolio usefulness

A target can be predicted well but still add little to a portfolio, while a weaker statistical model can still be useful if it gives a stable cross-sectional signal. The project therefore keeps separate values for:

- statistical target type,
- validation performance,
- final-test predictability,
- normalized Quality Score,
- fine-grained Portfolio Target Type,
- prediction horizon,
- horizon score,
- target-type value.

These are combined only in the portfolio layer.

2.4 Search efficiency without changing the test

The model and portfolio search spaces can be very large. The code reduces computation by:

- skipping completed model/parameter combinations,
- progressive model pruning,
- loading only selected features,
- caching predictions before repeated horizon backtests,
- freezing horizon parameters with low sensitivity,
- random-context pruning,
- exhaustive evaluation only when the remaining space is small enough.

The aim is to reduce runtime without changing what is being compared.

3 End-to-end architecture

The implemented architecture, including the optional intraday conversion step, is:

```
Equity Selector - Features & Targets Quant Reference
    |
    v
Data_Creation_Screening.py
- download OHLCV + ^GSPC
- screen stocks
- build individual, market-relative and cross-stock features
- target-blind feature screens
- build targets
- target-specific feature screens on training data
```

```

|
+--> Features_Targets_Data.db
+--> Selected_Features.txt
|
+--> [intraday tables only] Intraday Conversion.py
|   - infer lookbacks/horizons
|   - enforce session-safe windows
|   - update selected-feature mapping
v
Model_Fitting.py
- load one target + its selected features
- resume completed fold/configuration work
- chronological walk-forward racing
- promote finalists and complete every fold
- build Testing Eligible leaderboard
|
+--> Validation_Model_Fits/<STOCK_TYPE>.db
v
Best_Model_Test.py
- select one test-eligible model for each surviving target
- fit on train + validation, with target purge
- score final chronological test block
- calculate predictability / portfolio metadata
|
+--> Final_Test_Results.db
|   - Final Test Results <STOCK_TYPE>
|   - Most Predictable Results <STOCK_TYPE>
v
Horizon Score Backtests.py
- load portfolio-eligible model specifications
- train on earlier research dates
- predict once on horizon-validation dates
- search target-type x horizon score grid
- sensitivity freeze -> random-context screens -> exhaustive survivors
v
Backtest Database.py
- choose train/backtest tickers and date ranges
- download with 3-year feature warm-up
- rebuild only required features/targets
- cache selected model metadata and benchmark
|
+--> Backtest_Features_Targets.db
v
Backtest Simulations.py
- fit selected signal models once per universe scenario
- cache predictions
- score the fine-grained target types
- sweep portfolio settings and retained horizon-score configurations
- compare to S&P 500
|
+--> Automated_Backtest_Results.csv
+--> Automated_Backtest_Results.db
v
Precise Backtest.py
- rerun one exact configuration
- reconstruct held weights and P&L contributions
- compute risk, drawdown, anomaly and ADV/capacity diagnostics
|
+--> Single_Backtest_Anomalies.csv
+--> Single_Backtest_Anomaly_Report.db

```

Each script has a narrow role. Data creation decides what information is available. Validation finds model configurations worth testing. Final testing decides which target signals remain usable. Horizon research tests how much each target/horizon should matter. Broad simulation tests complete portfolio settings. The precise backtest explains selected results in detail.

3.1 Script roles and hand-offs

Script	Main role	Main inputs	Main outputs
<code>Data_Creation_Screening.py</code>	Daily data creation, stock/feature screening, target creation and target-specific screening	Yahoo Finance OHLCV, $\hat{G}SPC$, feature/target/screening packages	<code>Features_Targets_Data.db</code> , <code>Selected_Features.txt</code>
<code>IntradayConversion.py</code>	Convert intraday tables to session-safe lookback/horizon rules	Intraday stock-type table, selected-feature dictionary	Cleaned intraday table, updated selected features
<code>Model_Fitting.py</code>	Resume-aware walk-forward model search and finalist validation	Screened table and target-specific features	<code>Validation_Model_Fits/<STOCK_TYPE>.db</code>
<code>Best_Model_Test.py</code>	Final chronological model test and portfolio metadata	Test-eligible validation leaderboards	<code>Final_Test_Results.db</code>
<code>HorizonScoreBacktests.py</code>	Horizon/type importance research	Most Predictable Results, selected features, research table	Retained horizon-score configurations
<code>BacktestDatabase.py</code>	Build the train/backtest cache used by strategy research	Selected models/features, dates, tickers, Yahoo Finance	<code>Backtest_Features_Targets.db</code>
<code>BacktestSimulations.py</code>	Broad universe and portfolio sweep	Cached model data, benchmark, horizon configurations	<code>Automated_Backtest_Results.csv/.db</code>
<code>PreciseBacktest.py</code>	One exact strategy rerun with detailed diagnostics	Cached data and selected configuration	Anomaly CSV and detailed SQLite report

4 Quantitative definitions

4.1 Feature and target reference

The project keeps feature and target mathematics in *Equity Selector - Features & Targets Quant Reference*. The current reference contains 114 feature functions and 26 target functions. It separates rolling values, which use current and past data, from future values used only in targets. It also records the data requirements for cross-sectional and breadth calculations and the forward horizons that require purging in validation.

4.2 Feature families used in data creation

`Data_Creation_Screening.py` applies the feature library in dependency order. Per stock it calls:

1. `all_return_features`
2. `all_momentum_features`
3. `all_volatility_features`
4. `all_range_volatility_features`
5. `all_trend_features`
6. `all_moving_average_features`
7. `all_drawdown_features`
8. `all_distribution_features`
9. `all_tail_risk_features`
10. `all_volume_features`

11. `all_liquidity_features`
12. `all_ohlc_features`
13. `all_market_relative_features`
14. `all_beta_features`
15. `all_correlation_features`
16. `all_residual_features`
17. `all_technical_features`
18. `all_regime_features`
19. `all_interaction_features`
20. `all_composite_features`
21. `all_experimental_features`

For a multi-stock panel it then adds `all_cross_sectional_features`, builds wide market panels for `all_breadth_features` and `all_dispersion_features`, and merges the new market-level columns back to each ticker/date row.

The reference groups the calculations into beta, breadth, composites, correlation, cross-sectional ranking and normalization, dispersion, return distribution, drawdown, experimental persistence/entropy, interactions, liquidity, market-relative behaviour, momentum, moving averages, OHLC path structure, range volatility, regimes, residual behaviour, returns, sector-relative behaviour, tail risk, technical transforms, trend, volatility and volume.

4.3 Target families used in data creation

The target layer is created separately from the feature matrix. `Data_Creation_Screening.py` calls:

- `all_return_targets`,
- `all_volatility_targets`,
- `all_direction_targets`,
- `all_barrier_targets`,
- `all_excursion_targets`,
- `all_drawdown_targets`,
- `all_risk_adjusted_targets`,
- `all_ranking_targets` when the panel contains multiple stocks.

These cover forward returns, future realised risk, direction and threshold labels, path-dependent barriers, favourable/adverse excursions and timing, future downside paths, risk-adjusted outcomes and cross-sectional future-performance ranks.

5 Stage I - `Data_Creation_Screening.py`

5.1 Research universes

The data-creation script supports several liquidity and composition tests rather than one fixed ticker list.

Stock type	Size	Source definition	Purpose
High Liquidity 30	30	high_liquidity_30	Large, heavily traded equities
Medium Liquidity 30	30	medium_liquidity_30	Middle-liquidity comparison universe
Lower Liquidity 30	30	lower_liquidity_30	Lower-liquidity comparison universe
Sector Spread 30	30	sector_matched_liquidity_30	Sector-spread sample across liquidity levels
Liquidity Barbell 30	30	liquidity_barbell_30	15 highly liquid and 15 lower-liquidity names
Institutional Liquidity 60	60	institutional_liquidity_60	High plus medium liquidity names
Medium Small Liquidity 60	60	mid_small_liquidity_60	Medium plus lower-liquidity names
Medium Large Liquidity 60	60	mid_large_liquidity_60	Large/medium liquidity variant
All Liquidity 90	90	full_liquidity_spectrum_90	Full high/medium/lower spectrum
Intraday high-liquidity variant	30	–	Intraday-session research table
Intraday medium-liquidity variant	30	–	Intraday-session research table

The active selection in the supplied script is:

```
tokens = liquidity_barbell_30
```

The 60- and 90-stock universes use the same high-, medium- and lower-liquidity base groups so the feature and target pipeline can be compared across different tradability profiles.

5.2 Source history and benchmark

The daily download is:

```
yf.download(..., start="2013-01-01", end="2023-09-30", auto_adjust=True)
```

for the chosen stock universe, with a separate $\hat{G}$ SPC download over the same period. Stock data provides adjusted OHLCV fields. The benchmark is used by market-relative features and later benchmark comparisons.

5.3 Stock screening before feature engineering

Each ticker is screened before the main feature panel is built. The current stock-level sequence is:

1. `size_stocks(df)`
2. `missingness_stocks(df)`
3. `invalid_stocks(df)`
4. `continuity_stocks(df, market_df)`

Only stocks that pass these checks continue to feature creation.

5.4 Feature creation and target-blind feature screening

After feature generation, the global feature set is screened without using target performance. The package checks missingness, invalid values, zero variance, duplicate construction and high feature-to-feature correlation. This leaves a common feature pool before any feature is judged against a target.

5.5 Target creation and chronological split

Targets are appended after the first feature panel is built and screened. The complete dataframe is passed to `train_validation_test_split(df)`, which creates chronological train, validation and test partitions. Target-specific feature screening uses only `train_df`.

5.6 Target-specific feature screens

Each target starts with a copy of the target-blind feature list. The target type is inferred from the training data and the following screens are applied:

Target type	Target-specific screens
All types	Feature-target coverage, then time stability
Continuous	Quantile spread, quantile monotonicity, Pearson correlation, IC screening when the universe has at least 30 stocks
Binary	Quantile spread, quantile monotonicity, Pearson correlation
Multiclass	Universal coverage and time-stability screens

The result is a dictionary such as:

```
{
  "Target A": ["Feature 1", "Feature 7", "Feature 21"],
  "Target B": ["Feature 3", "Feature 5"],
}
```

One dictionary line is stored per stock type in `Selected_Features.txt`. Later scripts use the stock-type line to recover the correct target-to-feature mapping.

5.7 Stored research table

Only features used by at least one target are kept in the final screened panel. The table contains date/ticker identity, OHLCV fields needed later, all target columns and the union of target-selected features. The supplied active run writes the `Liquidity Barbell 30` table to `Features_Targets_Data.db`. Other stock types use their own table names in the same database.

5.8 Intraday session conversion - Intraday Conversion.py

Intraday tables use an extra session-aware conversion stage. The script identifies base columns, target columns, feature lookbacks, target lookbacks and target forward horizons from column names. Manual override dictionaries are available where a name does not fully describe the required lookback or horizon.

The current intraday cap is `MAX_PERIODS = 60`. Features with longer inferred lookbacks and targets with longer lookback/horizon requirements are removed from the intraday table and from the matching line in `Selected_Features.txt`.

The conversion is session-aware by ticker and calendar date. Rolling features are unavailable during their start-of-session warm-up. Targets are unavailable during their own warm-up and across the final horizon rows of the session, so a forward label does not cross into the next trading session. The first

intraday return of each session is also unavailable. The cleaned table replaces the original intraday table while keeping the row structure.

6 Stage II - Model_Fitting.py

6.1 Target-specific loading

Model_Fitting.py first reads the chosen stock-type schema. For each target it loads only:

Date, Ticker, current target, selected features for current target

from Features_Targets_Data.db. The full wide table is not held in memory for every target.

The active validation database is stock-type specific: Validation_Model_Fits/<STOCK_TYPE>.db, with spaces in the stock-type name replaced by underscores. The validation window is 20 chronological dates and the shared random state is 42.

6.2 Active model catalogue

Target type	Active models	Search style
Continuous	Mean Baseline, OLS, Ridge, Lasso, XGBoost	Mean/OLS/Ridge/Lasso: grid; XGBoost: random (5 draws)
Binary	Binary Baseline, Logistic Regression, L2 Logistic Regression, XGBoost, Naive Bayes	Baseline/logistic/Naive Bayes: grid; XGBoost: random (5 draws)
Multiclass	Multiclass Baseline, Multinomial Logistic Regression, L2 Multinomial Logistic Regression, XGBoost, Ordinal Regression	Baseline/logistic/ordinal: grid; XGBoost: random (5 draws)

The Ridge grid uses alphas 0.0001, 0.0003, 0.001, 0.01, 0.1, 1, 10, 100 and 1000. The Lasso grid uses 0.01, 0.1, 1 and 10. The L2 logistic grids use $C = 0.0001, 0.0003, 0.001, 0.01, 0.1, 1, 10$ and 100. Logistic models also support no class weighting and balanced class weighting. XGBoost samples five random parameter configurations per active search definition.

The script also contains prepared definitions that can be enabled for Elastic Net, Huber, histogram gradient boosting, standard gradient boosting, random forest, LightGBM, support-vector models, neural networks, k-nearest-neighbour models and additional classification families. These use the same validation and database process.

6.3 Resume and configuration history

For each target, the validation database stores exact model/parameter/fold results. Existing rows are loaded before new work starts, so completed configurations and folds are reused after a restart or model-catalogue change.

The main table roles are:

- <target>__folds - complete fold-level history,
- <target>__search - search/racing leaderboard,
- <target> - final all-fold leaderboard used for final-test selection.

6.4 Chronological walk-forward folds

The model-development train and validation blocks are combined into a chronological development dataset. Validation dates are split into consecutive 20-date blocks. For each fold:

- the validation block is a later date range,
- training uses only observations before that block,
- `target_purge_days(target)` removes overlapping training dates before validation,
- any scaler is fitted only on the current training fold and applied to validation afterwards.

6.5 Progressive racing schedule

No performance pruning is done before fold 3. After each listed stage, configurations are compared within their model family.

Completed fold stage	Keep fraction	Minimum per family
3	50%	5
5	50%	3
7	50%	2

A family with one configuration left is kept. Future Return Rank continuous targets use Rank IC as the main metric. Other continuous targets use RMSE, binary targets use ROC AUC, and multiclass targets use Macro F1.

6.6 Finalists and full validation

Configurations with at least three successful search folds are treated as mature candidates. Their ranking includes a $0.25\times$ standard-deviation penalty so both average performance and fold variability matter.

Up to 8 mature finalists per target are promoted across model families. Up to 2 partial legacy candidates per model family can also be promoted where required. Every finalist is then run on every missing walk-forward fold, reusing exact results already stored. The final target leaderboard therefore compares finalists over the same complete fold schedule.

6.7 Leaderboard metrics

Successful folds are aggregated into mean metrics, cross-fold standard deviations and Fold Count. Sorting is target-aware:

- Future Return Rank continuous targets: Rank IC mean descending, Rank IC standard deviation ascending, then RMSE/MAE stability and level.
- Other continuous targets: RMSE mean and standard deviation ascending, MAE ascending, then R^2 and Rank IC descending.
- Binary targets: ROC AUC mean descending/stability ascending, PR AUC descending/stability ascending, Log Loss ascending, F1 descending.
- Multiclass targets: Macro F1 descending/stability ascending, Balanced Accuracy descending/stability ascending, Log Loss ascending.

6.8 Exact Testing Eligible gates

Every final-test candidate must first have `Fold Count == total_folds`. The supplied gates are:

Continuous

$$[(R^2 \geq 0.05) \wedge (|IC| \geq 0.10)] \vee (|IC| \geq 0.20), \quad \text{Rank IC Std} \leq 0.10.$$

Binary

$$ROC\ AUC \geq 0.60, \quad PR\ AUC \geq 0.20, \quad \text{Balanced Accuracy} > 0.50,$$

with `ROC AUC Std` ≤ 0.07 .

Multiclass

$$Macro\ F1 \geq 0.45,$$

with `Macro F1 Std` ≤ 0.10 .

The final leaderboard stores `Testing Eligible` and a readable reason for the result.

7 Stage III - Best_Model_Test.py

7.1 Test-eligible model selection

The script loads the final target leaderboards from the stock-type validation database and keeps rows marked `Testing Eligible`. For each target:

- no eligible row means the target does not enter the final test,
- one eligible row is selected automatically,
- if there are several eligible rows, the script prints model, parameters, rank and relevant validation metrics so one is selected for the final test.

The displayed metrics are continuous R^2 Mean, Rank IC Mean, RMSE Mean and Rank IC Std; binary ROC AUC Mean, PR AUC Mean, F1 Mean and ROC AUC Std; and multiclass Macro F1 Mean, Balanced Accuracy Mean, Log Loss Mean and Macro F1 Std where available.

7.2 Final chronological fit/test

Only the current target and its selected features are loaded from SQLite. The original train/validation/test split is rebuilt. The final training sample is:

```
final_train_df = concat(train_df, validation_df)
```

and `target_purge_days(target)` is applied immediately before fitting. The selected estimator is then evaluated once on `test_df`. In the 60/20/20 development split, model selection uses the first 80% as train plus validation and the final 20% is the unseen model-test block.

7.3 Stored final-test metrics

Statistical target type	Stored test metrics
Continuous	RMSE, MAE, R^2 , Rank IC
Binary	ROC AUC, PR AUC, Log Loss, F1
Multiclass	Macro F1, Balanced Accuracy, Log Loss

Each row also stores target, target type, selected model and JSON-serialized parameters in `Final Test Results <STOCK_TYPE>` inside `Final_Test_Results.db`.

8 From final test to portfolio metadata

8.1 Final-test retention and Predictability Score

The final-test table contains one selected model per tested target. Baseline rows themselves are not used as portfolio signals. A target is retained for portfolio research when its selected model beats the appropriate baseline for its statistical type (the Mean Baseline for continuous targets). Weak targets are not given extra portfolio influence simply because they remain in the table; their Quality Score can reduce their contribution to zero or close to zero.

The Predictability Score is still stored as a compact summary of final-test evidence:

Continuous

$$P = 0.65|IC| + 0.35 \max(R^2, 0).$$

Binary

$$A = 2 \max(ROC \ AUC - 0.5, 0),$$
$$P = 0.50A + 0.30(PR \ AUC) + 0.20F1.$$

Multiclass

$$P = Macro \ F1.$$

Because only one final-tested model is carried for each target, Predictability Score is descriptive rather than a second model-selection step. The Quality Score is the value used later to control how strongly a target can affect the portfolio.

8.2 Fine-grained Portfolio Target Type

Each target is mapped directly to a fine-grained economic type. The current classifier can produce:

ALPHA	RELATIVE_ALPHA	CROSS_SECTION_ALPHA
DIRECTION	ALPHA_BINARY	BARRIER_ALPHA
VOLATILITY	VOLATILITY_EVENT	DOWNSIDE
TAIL_RISK	TAIL_EVENT	UPSIDE_RISK
UPSIDE_EVENT	RECOVERY	REVERSAL
REGIME	CORRELATION	COVARIANCE
LIQUIDITY	EXECUTION	MARKET_IMPACT

This classification is used directly in horizon research and portfolio scoring. The old three-channel Alpha/Volatility/Downside layer is not used in the portfolio calculation.

8.3 Horizon extraction

The target horizon is stored explicitly. Names can use minutes, hours, days or weeks. If there is no unit, the final standalone number in the target name is used, while numbers inside percentage thresholds are ignored. `Horizon Score Backtests.py` uses this value together with Portfolio Target Type to select the relevant horizon-score parameter.

8.4 Cross-target Quality Score

Quality Score puts model evidence on a common clipped $[0, 1]$ scale.

For continuous targets:

$$q_{IC} = \text{clip}(|IC|/0.30), \quad q_{R^2} = \text{clip}(\max(R^2, 0)/0.20),$$

$$Q_{model} = 0.70q_{IC} + 0.30q_{R^2},$$

using whichever component metrics are available.

For binary targets:

$$q_{ROC} = \text{clip}((ROC\ AUC - 0.50)/0.25).$$

When event prevalence p is available, PR AUC is normalized relative to the base rate and an excellent level of $\min(1, p + 0.30)$. Event-like types `TAIL_EVENT`, `VOLATILITY_EVENT` and `UPSIDE_EVENT` use 60% PR quality and 40% ROC quality. Other binary types use 60% ROC quality and 40% PR quality.

For multiclass targets, Macro F1 is normalized from chance level $1/K$ to an excellent level of 0.70 and receives 70% weight. Multiclass AUC, when available, supplies the remaining 30% after normalization from 0.50 to 0.75.

The portfolio-facing table is `Most Predictable Results <STOCK_TYPE>`. It stores target, model, parameters, statistical target type, Predictability Score, Portfolio Target Type, Horizon and Quality Score.

9 Stage IV - Horizon Score Backtests.py

9.1 Research-period role

This script performs horizon-score research inside the pre-2023-09-30 research block. `RESEARCH_END` is 2023-09-30 and the final 25% of available research dates is used as the horizon-score validation block. Earlier research dates fit the selected models and the later quarter compares horizon-score configurations. Only targets that exist in both `Most Predictable Results <STOCK_TYPE>` and the selected-feature dictionary are loaded. Baseline model rows are excluded.

9.2 Predict once, vary horizon scores

For every selected target, the script loads only the required dates and columns, fits the chosen model on the horizon training block and writes predictions into one common `predictions_df`. Predictions are not regenerated for every horizon-score combination. The search changes only the horizon weighting metadata used by the portfolio backtest.

9.3 Horizon-score catalogue

The code defines 21 Portfolio Target Types and 10 canonical horizon keys: 1m, 5m, 15m, 60m, 1d, 5d, 20d, 60d, 120d and 252d. This gives 210 possible type-horizon cells. Only cells represented by the surviving target set become active parameters.

Every active range is discretized in steps of 0.05. Representative endpoints from the configured catalogue are shown below. The complete code contains all ten horizon ranges for every type.

Portfolio Target Type	1m	1d	20d	252d
ALPHA	0.00–0.20	0.25–0.55	0.70–0.95	0.45–0.80
RELATIVE_ALPHA	0.00–0.15	0.20–0.50	0.75–1.00	0.50–0.85
CROSS_SECTION_ALPHA	0.00–0.15	0.25–0.55	0.85–1.00	0.35–0.65
DIRECTION	0.10–0.40	0.60–0.90	0.75–1.00	0.05–0.35
ALPHA_BINARY	0.20–0.50	0.70–1.00	0.70–1.00	0.05–0.35
BARRIER_ALPHA	0.25–0.55	0.70–1.00	0.75–1.00	0.05–0.35
VOLATILITY	0.15–0.45	0.65–0.95	0.80–1.00	0.25–0.55
VOLATILITY_EVENT	0.50–0.80	0.80–1.00	0.40–0.70	0.00–0.20
DOWNSIDE	0.25–0.55	0.75–1.00	0.75–1.00	0.15–0.45

Portfolio Target Type	1m	1d	20d	252d
TAIL_RISK	0.40–0.70	0.85–1.00	0.60–0.90	0.00–0.30
TAIL_EVENT	0.85–1.00	0.60–0.90	0.10–0.40	0.00–0.10
UPSIDE_RISK	0.20–0.50	0.70–1.00	0.70–1.00	0.05–0.35
UPSIDE_EVENT	0.85–1.00	0.55–0.85	0.05–0.35	0.00–0.10
RECOVERY	0.85–1.00	0.50–0.80	0.00–0.30	0.00–0.05
REVERSAL	0.85–1.00	0.50–0.80	0.00–0.30	0.00–0.05
REGIME	0.00–0.25	0.35–0.65	0.85–1.00	0.45–0.75
CORRELATION	0.00–0.25	0.40–0.70	0.85–1.00	0.45–0.75
COVARIANCE	0.00–0.25	0.40–0.70	0.85–1.00	0.45–0.75
LIQUIDITY	0.65–0.95	0.55–0.85	0.05–0.35	0.00–0.05
EXECUTION	0.85–1.00	0.25–0.55	0.00–0.15	0.00–0.00
MARKET_IMPACT	0.85–1.00	0.25–0.55	0.00–0.15	0.00–0.00

The start value for each active parameter is candidate index 2 (HORIZON_INDEX = 2), clipped to the final valid index if a list has fewer values.

9.4 Fixed portfolio settings during horizon research

Every horizon configuration is compared with the same portfolio settings:

```
rebalance_every = 1
max_weight = 0.30
concentration_penalty = 0.10
trading_fee = 0.00
```

This keeps the horizon-score test separate from the later portfolio-parameter sweep.

9.5 Benchmark-relative Backtest Quality

The benchmark is evaluated on the same horizon-validation dates. Four differences are calculated:

- ΔS : strategy Sharpe minus benchmark Sharpe,
- ΔR : strategy total return minus benchmark total return,
- ΔD_{max} : absolute benchmark maximum drawdown minus absolute strategy maximum drawdown,
- ΔD_{avg} : absolute benchmark average drawdown minus absolute strategy average drawdown.

The components are bounded before they are combined:

$$BQ = 0.40 \tanh\left(\frac{\Delta S}{0.50}\right) + 0.30 \tanh\left(\frac{\Delta R}{0.10}\right) + 0.20 \tanh\left(\frac{\Delta D_{max}}{0.10}\right) + 0.10 \tanh\left(\frac{\Delta D_{avg}}{0.05}\right).$$

Sharpe improvement has the largest weight, followed by return, maximum-drawdown improvement and average-drawdown improvement.

9.6 One-dimensional sensitivity freeze

All other active horizon parameters are first held at their index-2 values while one parameter is swept through its candidates. For that parameter the code calculates the mean and standard deviation of Backtest Quality and then:

$$CV = \frac{\sigma(BQ)}{|\mu(BQ)|}.$$

If $CV < 0.05$, the parameter is treated as insensitive and is fixed at its index-2 value for later stages.

9.7 Random-context screens

Each parameter that is still variable is compared under the same random background configurations for every candidate value. This produces a head-to-head win rate for each candidate.

Round	Random backgrounds	Minimum win rate kept
1	20	0.15
2	50	0.30
3	100	0.35

The best candidate for each parameter is kept even if no value meets the threshold. Frozen parameters remain fixed. If more than 1,000 complete configurations remain after the scheduled rounds, the script can request a larger iteration count and stronger threshold and repeat the random-context screen. An explicit override is also available so the user can stop further pruning and move on with more than 1,000 remaining configurations when that is preferred.

9.8 Exhaustive survivor search

The Cartesian product of the remaining horizon-score candidate lists is evaluated exhaustively. Configurations with $BQ < 0$ are not retained. Non-negative survivors are sorted by Backtest Quality and the top 5% are kept, with at least one retained configuration when survivors exist.

10 Stage V - Backtest Database.py

10.1 Purpose

`Backtest Database.py` creates a self-contained strategy-research cache for a chosen training universe, backtest universe and chronological date range. It reuses the selected model/target metadata from `Final_Test_Results.db` and the stock-type target-to-feature mapping from `Selected_Features.txt`.

10.2 User-specified research windows

The script prompts for:

- training tickers,
- training start and end dates,
- backtest tickers,
- backtest start and end dates,
- default rebalance frequency (default 60 trading days).

The training and backtest periods are kept chronologically separate.

10.3 Three-year feature warm-up and one market download

`FEATURE_WARMUP_YEARS = 3`. Raw data therefore starts three years before the earliest requested modelling date so rolling features have enough history. All requested train and backtest tickers are downloaded together, and `̂SPC` is downloaded as the benchmark.

The feature panel is rebuilt in the same dependency order used during development. Only the union of features required by the selected targets/models is needed in the cached database. Selected target functions are rebuilt on the same price panel.

10.4 Cached database schema

`Backtest_Features_Targets.db` contains:

- `Model_Data` - training and backtest rows with a Split label,
- `Benchmark` - S&P 500 benchmark history,
- `Selected Models <STOCK_TYPE>` - selected target/model/parameter and portfolio metadata,
- `Selected Model Features <STOCK_TYPE>` - target-specific feature lists,
- `Required_Features` - union of features required by the selected models,
- `Config` - stock type, ticker sets, dates, rebalance choice, warm-up period, counts, table names and creation timestamp.

Indexes on (`Split`, `Date`, `Ticker`) and benchmark `Date` support repeated downstream reads. The broad research run can end around March 2025 so later data remains available for a final strategy holdout.

11 Stage VI - Backtest Simulations.py

11.1 Fine-grained portfolio signal structure

The broad simulation uses the fine-grained Portfolio Target Type directly. For each selected target j and stock i , the model output is converted to a comparable target signal S_{ij} . The target then receives its tested horizon score H_j and model Quality Score Q_j .

The model weight is:

$$M_j = H_j Q_j,$$

and the target contribution for stock i is:

$$C_{ij} = S_{ij} H_j Q_j.$$

Targets that share the same fine-grained Portfolio Target Type g are combined using a horizon-quality weighted mean:

$$T_{ig} = \frac{\sum_{j \in g} S_{ij} H_j Q_j}{\sum_{j \in g} H_j Q_j}.$$

Each target type has a configured value V_g . Positive values reward desirable types and negative values penalize risk-type signals. Its type contribution is:

$$\text{TypeContribution}_{ig} = V_g T_{ig}.$$

The overall stock score is normalized by the absolute target-type values:

$$\text{Score}_i = \frac{\sum_g V_g T_{ig}}{\sum_g |V_g|}.$$

Equivalently,

$$\text{Score}_i = \frac{\sum_g V_g \left[\frac{\sum_{j \in g} S_{ij} H_j Q_j}{\sum_{j \in g} H_j Q_j} \right]}{\sum_g |V_g|}.$$

For each universe scenario, the selected predictive models are fitted once on the training tickers and predictions are generated once for the backtest rows. The parameter sweep reuses these predictions and changes the scoring and portfolio settings without refitting every model for every grid row.

11.2 Universe scenarios

The Manual setup now uses the stocks in `STOCK_TYPE` as the default training universe. Its test universe is made from similar stocks that are not already in that `STOCK_TYPE`, so the test checks transfer to comparable unseen securities rather than reusing the development names.

The other cached-universe mechanisms remain available:

- **All Cached** - use the complete cached train and backtest stock sets,
- **Relative liquidity universes** - Low, Medium and High buckets built from the cached stocks.

Liquidity is measured by median dollar volume:

$$DollarVolume_{i,t} = Close_{i,t} \times Volume_{i,t}.$$

Training-stock liquidity is calculated from training data. Backtest-stock liquidity uses only the first 60 backtest trading days (`LIQUIDITY_CLASSIFICATION_DAYS` = 60). The strategy period starts after that classification window. Ranked stocks are split into three equally sized relative-liquidity buckets.

11.3 Portfolio optimizer

At a rebalance date, the optimizer uses the stock scores defined above and applies a quadratic concentration penalty:

$$w^* = \arg \max_w \left[\sum_i w_i \text{Score}_i - \lambda \sum_i w_i^2 \right]$$

subject to

$$\sum_i w_i = 1, \quad 0 \leq w_i \leq w_{max}.$$

The default values are $\lambda = 0.10$ and $w_{max} = 0.30$. The numerical optimizer is SciPy SLSQP and starts from equal weights. A rebalance is skipped if the available cross-section cannot satisfy the fully invested maximum-weight constraint.

11.4 Parameter sweep

The active broad sweep uses the configured fine-grained target-type values V_g rather than the old three-channel Alpha/Volatility/Downside importance presets. The non-horizon portfolio settings continue to include:

- maximum position weight: 0.10, 0.15, 0.20, 0.25, 0.30,
- concentration penalty: 0.00, 0.05, 0.10, 0.20,
- rebalance frequency: 20, 40, 60, 90, 120 trading days,
- Trading 212 commission assumption: `TRADING_FEE` = 0.00.

The active non-horizon setup contains up to 2,500 universe/portfolio configurations. Every retained horizon-score configuration from Stage IV is also run through the sweep. Therefore the intended simulation count is:

$$N_{sim} = 2500 \times N_{horizon \text{ configurations}}.$$

This count can be reduced if the full run takes too long.

11.5 Position timing and P&L accounting

Portfolio weights are generated on rebalance rows, forward-filled between rebalances and shifted by one row before daily returns are applied:

```
held_weights = weights_df.shift(1)
```

This is the portfolio held for the following row's return. Daily ticker contribution is held weight times stock return. Strategy return is the cross-sectional sum less the configured turnover-based trading cost. Daily turnover is the sum of absolute changes in weights. The S&P 500 path is normalized to the same strategy start for direct comparison.

11.6 Stored experiment fields and summaries

Each result row stores universe identity, train/test tickers, fine-grained target-type values, horizon-score configuration, portfolio settings, strategy and S&P 500 returns, relative performance, annualized mean and volatility, Sharpe measures, drawdowns, position changes, average holdings, average turnover, skipped rebalances, strategy dates, final rebalance and final holdings.

The full sweep is written to `Automated_Backtest_Results.csv` and `Automated_Backtest_Results.db`. The SQLite output contains the full result table and grouped views by the main simulation dimensions so parameter behaviour can be judged across many runs rather than from one winner.

12 Selecting simulations for precise testing

The broad simulation stage is used to build a short list of candidates for deeper testing, not to take the single highest Sharpe row automatically. Candidate selection focuses on evidence that a hedge fund or large trading firm would want to inspect before treating a backtest as serious research.

Priority candidates include:

- strong benchmark-relative return and Sharpe without relying on one short period,
- controlled maximum and average drawdown,
- stable behaviour across nearby parameter and horizon-score configurations,
- acceptable concentration and effective number of holdings,
- turnover and rebalance behaviour that remain practical,
- position and trade sizes that remain reasonable relative to ADV,
- performance that is not driven by one stock or a very small number of days,
- results that transfer across relevant unseen stocks or universe variants,
- deliberately weak or unusual cases that are useful for checking failure modes.

The selected row gives `Precise Backtest.py` an exact universe, fine-grained target-type value set, horizon-score configuration and portfolio setting. The next stage reconstructs and explains that exact result.

13 Stage VII - Precise Backtest.py

13.1 Exact chosen configuration

`Precise Backtest.py` runs one supplied configuration. An exact run stores the chosen universe, fine-grained target-type values, maximum weight, concentration penalty, rebalance frequency, trading fee and the full horizon-score configuration used by the simulation.

A horizon-score configuration contains up to 210 target-type/horizon cells. The table below shows an abridged random example of the stored form; the full configuration also contains the omitted target types and the remaining horizon keys.

Portfolio Target Type	1m	1d	20d	252d
ALPHA	0.10	0.40	0.80	0.60
RELATIVE_ALPHA	0.05	0.35	0.90	0.70
CROSS_SECTION_ALPHA	0.10	0.45	0.95	0.50
DIRECTION	0.25	0.75	0.85	0.20
VOLATILITY	0.30	0.80	0.90	0.40
TAIL_RISK	0.55	0.95	0.75	0.15
...				
EXECUTION	0.90	0.40	0.10	0.00
MARKET_IMPACT	0.95	0.30	0.05	0.00

The Manual, All Cached and Low/Medium/High universe mechanics remain available, including the 60-day liquidity-classification window for liquidity-specific runs.

13.2 Daily diagnostic panel

After the exact backtest is rebuilt, the script creates daily diagnostics for:

- strategy return,
- S&P 500 return,
- active return,
- strategy-return z-score using only previous rolling history,
- active-return z-score using only previous observations,
- rolling annualized volatility and volatility/baseline ratio,
- rolling market beta,
- rolling market correlation,
- rolling Sharpe,
- turnover,
- weight sum,
- largest single weight,
- top-three aggregate weight,
- Herfindahl-Hirschman Index (HHI),
- effective holdings $1/HHI$,
- number of non-negligible holdings,
- drawdown,
- maximum position as a fraction of ADV,
- maximum trade as a fraction of ADV,
- largest absolute single-name P&L contribution share.

13.3 Current diagnostic thresholds

Diagnostic setting	Current value
Return shock rolling window	60 trading days
Return shock z threshold	3 z
Absolute daily return threshold	5.0%
Active-return z threshold	3 z
Absolute active-return threshold	4.0%
Short volatility window	20 trading days
Volatility baseline window	252 trading days
Volatility spike multiplier	1.75x
Beta window	60 trading days
Absolute beta threshold	1.5
Correlation window	60 trading days
Absolute correlation threshold	0.9
Rolling Sharpe window	60 trading days
Rolling Sharpe collapse	-1
Drawdown reporting threshold	-10%
Turnover absolute threshold	1
Turnover z threshold	3 z
Minimum effective holdings	3.5
Top-3 weight threshold	80%
P&L contribution concentration	75%
Minimum material daily P&L	1.0%
Loss-streak threshold	5 days
Capacity reference capital	10,000,000
ADV lookback	20 trading days
Position/ADV warning	1.0%
Trade/ADV warning	5.0%

A full change from one 100%-weighted portfolio to a different 100%-weighted portfolio is turnover 2.0 under the script's `sum(abs(change in weights))` definition.

13.4 Drawdown episodes and extreme days

The script rebuilds complete drawdown episodes from peak through trough to recovery, or to the end of the sample if still underwater. For each reportable episode it stores peak date, trough date, recovery date, depth, trading days to trough/recovery, the actual weights held at the trough and the largest P&L contributions around the event.

It also stores the 10 worst and 10 best strategy days with strategy return, S&P 500 return, active return, return z-score, drawdown, held weights and major ticker-level contributions.

13.5 Anomaly engine

The anomaly layer identifies:

- Daily Return Shock,
- Market-Relative Return Shock,
- Drawdown Trough,
- Volatility Regime Spike,

- Market Beta Spike,
- Market Correlation Spike,
- Rolling Sharpe Collapse,
- Turnover Spike,
- Low Effective Diversification,
- Top-3 Concentration,
- MAX_WEIGHT Constraint Breach,
- Portfolio Weight Integrity event,
- Single-Name P&L Concentration,
- Position Capacity Warning,
- Execution Capacity Warning,
- Missing Held Return,
- Loss Streak.

Contiguous regime masks are grouped into one episode and represented by their most extreme date. Each anomaly can store its event period, held weights, most recent rebalance, top P&L contributions and severity.

13.6 Full risk summary

The precise backtest also creates a risk table containing:

- total and annualized return,
- annualized volatility,
- Sharpe,
- downside deviation,
- Sortino,
- maximum drawdown,
- Calmar,
- daily VaR 95% and 99%,
- daily Expected Shortfall 95% and 99%,
- market beta and correlation,
- tracking error and information ratio,
- daily hit rate,
- daily skew and excess kurtosis,
- worst and best day,
- average/minimum effective holdings,
- maximum single position,
- average/maximum daily turnover,
- maximum position % ADV,
- maximum trade % ADV,

- count of missing held-return observations.

A separate year-by-year table stores strategy return, S&P 500 return, excess return, volatility, Sharpe, maximum drawdown and extreme daily returns for each year in the backtest.

14 End of coded research

From here, the existing pipeline uses fixed settings; strategy review and selection are manual.

15 Storage architecture

The databases store the exact outputs needed by the next stage.

15.1 Features__Targets__Data.db

Contains stock-type-specific screened feature/target panels. It is the source of the model-development data after stock screening, feature construction, target-blind screening, target creation and target-specific feature reduction.

15.2 Selected__Features.txt

Contains one stock-type-specific Python dictionary per line mapping each target to the feature columns allowed to model it. The same stock-type line index is reused by model fitting, final testing, horizon research, backtest-database construction and intraday conversion.

15.3 Validation__Model__Fits/<STOCK_TYPE>.db

For each target the database can contain:

- <target>__folds - complete walk-forward fold history,
- <target>__search - search/racing leaderboard,
- <target> - complete finalist leaderboard with **Testing Eligible**.

15.4 Final__Test__Results.db

Contains:

- **Final Test Results** <STOCK_TYPE> - selected models' final chronological test metrics,
- **Most Predictable Results** <STOCK_TYPE> - one retained row per target that beats its baseline, with Predictability Score, Portfolio Target Type, Horizon and Quality Score.

15.5 Backtest__Features__Targets.db

Contains the self-contained strategy cache:

- **Model_Data**,
- **Benchmark**,
- **Selected Models** <STOCK_TYPE>>,
- **Selected Model Features** <STOCK_TYPE>>,
- **Required_Features**,
- **Config**.

15.6 Automated_Backtest_Results.db

Contains the full simulation sweep plus grouped analyses across universe, maximum weight, rebalance frequency, concentration penalty, fine-grained target-type values and horizon-score configuration. The same run also writes `Automated_Backtest_Results.csv`.

15.7 Single_Backtest_Anomaly_Report.db

The precise-backtest database contains:

- `Backtest_Summary`,
- `Risk_Summary`,
- `Daily_Diagnostics`,
- `Daily_Held_Weights`,
- `Daily_Contributions`,
- `Rebalance_Risk`,
- `Anomalies`,
- `Anomaly_Weights`,
- `Anomaly_Period_Rebalances`,
- `Drawdown_Episodes`,
- `Loss_Streaks`,
- `Worst_Days`,
- `Best_Days`,
- `Yearly_Performance`.

A companion `Single_Backtest_Anomalies.csv` stores the anomaly event table outside SQLite.

16 Intended research sequence

1. Maintain and review the mathematical feature and target definitions in the quant reference.
2. Choose a research stock type and run `Data_Creation_Screening.py` over the development history.
3. Screen stocks before feature construction, then build individual, market-relative and cross-stock features.
4. Apply target-blind feature screens.
5. Build target families and the chronological train/validation/test split.
6. Apply target-specific screens only on the training block and write the stock-type line to `Selected_Features.txt`.
7. For an intraday stock type, run `Intraday_Conversion.py` to enforce session-safe lookbacks/horizons and update the same feature mapping.
8. Run `Model_Fitting.py`; resume existing fold history, race configurations, promote finalists and complete walk-forward validation.
9. Use final validation leaderboards and `Testing_Eligible` flags as the source of final-test candidates.
10. Run `Best_Model_Test.py` and select one eligible model for each tested target.

11. Fit the selected model on train plus validation with the target purge and score the final test block.
12. Build **Most Predictable Results** <STOCK_TYPE> with the retained target, fine-grained Portfolio Target Type, Horizon, Predictability Score and Quality Score.
13. Run **Horizon Score Backtests.py**: calculate predictions once, freeze insensitive cells, run random-context screens, optionally override the 1,000-configuration pruning target, exhaustively test the chosen survivor space and retain the top non-negative 5%.
14. Use **Backtest Database.py** to build the later train/backtest cache with three years of raw feature warm-up.
15. Run **Backtest Simulations.py** over the enabled universes, fine-grained target-type values, portfolio settings and every retained horizon-score configuration.
16. Use full and grouped results to select robust, explainable candidates for detailed testing.
17. Run **Precise Backtest.py** for each chosen exact configuration and inspect holdings, contributions, risk, drawdowns, anomalies, capacity and yearly behaviour.

17 What counts as evidence at each layer

Layer	Question	Main evidence
Quant reference	What is each input/target mathematically?	Formula, interpretation, data requirements and output definition
Stock screening	Is the security suitable for this stock-type panel?	Size, missingness, invalid values and continuity
Target-blind feature screening	Is the feature structurally suitable before target performance is used?	Missingness, invalid values, variance, duplicates and correlation structure
Target-specific screening	Could the feature help this target?	Coverage, quantile spread/monotonicity, Pearson, IC where relevant and time stability
Walk-forward search	Which model/parameter sets stay competitive through time?	Fold metrics, fold variability and racing history
Final validation leaderboard	Which candidates deserve the unseen model test?	Complete Fold Count and target-specific Testing Eligible gate
Final model test	Did the selected model survive unseen model data?	RMSE/MAE/ R^2 /Rank IC, ROC/PR/Log Loss/F1, or Macro F1/Balanced Accuracy/Log Loss
Portfolio target layer	Which tested targets enter portfolio research and how strongly?	Baseline comparison, Predictability Score, fine-grained Portfolio Target Type, Horizon and Quality Score
Horizon research	Which type/horizon regions improve benchmark-relative behaviour?	Backtest Quality, sensitivity CV, random-context win rates and exhaustive survivor ranking
Broad simulation	Which complete allocation settings deserve more work?	Cross-universe return, Sharpe, drawdown, turnover, concentration, stability, capacity and horizon-configuration results
Precise backtest	Why did a chosen strategy behave that way?	Held weights, ticker contributions, drawdown episodes, risk summary, anomalies and capacity

A Key formula summary

A.1 Predictability Score

Continuous:

$$P_c = 0.65|IC| + 0.35 \max(R^2, 0).$$

Binary:

$$P_b = 0.50[2 \max(AUC - 0.5, 0)] + 0.30(PR \ AUC) + 0.20F1.$$

Multiclass:

$$P_m = Macro \ F1.$$

A.2 Quality Score

Continuous:

$$q_{IC} = \text{clip}(|IC|/0.30), \quad q_{R^2} = \text{clip}(\max(R^2, 0)/0.20),$$

$$Q_{model} = 0.70q_{IC} + 0.30q_{R^2}.$$

Binary and multiclass Quality Scores use the normalized ROC/PR and Macro F1/AUC rules described in Section 8.4.

A.3 Backtest Quality used by horizon research

$$BQ = 0.40 \tanh(\Delta S/0.50) + 0.30 \tanh(\Delta R/0.10) + 0.20 \tanh(\Delta D_{max}/0.10) + 0.10 \tanh(\Delta D_{avg}/0.05).$$

A.4 Horizon sensitivity ratio

$$CV = \frac{\sigma(BQ)}{|\mu(BQ)|},$$

with sensitivity freeze at $CV < 0.05$.

A.5 Fine-grained target model weight and contribution

For target j :

$$M_j = H_j Q_j,$$

$$C_{ij} = S_{ij} H_j Q_j.$$

The fine-grained Portfolio Target Types are:

ALPHA	RELATIVE_ALPHA	CROSS_SECTION_ALPHA
DIRECTION	ALPHA_BINARY	BARRIER_ALPHA
VOLATILITY	VOLATILITY_EVENT	DOWNSIDE
TAIL_RISK	TAIL_EVENT	UPSIDE_RISK
UPSIDE_EVENT	RECOVERY	REVERSAL
REGIME	CORRELATION	COVARIANCE
LIQUIDITY	EXECUTION	MARKET_IMPACT

A.6 Combine targets inside one type

$$T_{ig} = \frac{\sum_{j \in g} S_{ij} H_j Q_j}{\sum_{j \in g} H_j Q_j}.$$

A.7 Apply target-type value

$$\text{TypeContribution}_{ig} = V_g T_{ig}.$$

Positive V_g rewards desirable signals and negative V_g penalizes risk-type signals.

A.8 Overall stock score

$$\text{Score}_i = \frac{\sum_g V_g T_{ig}}{\sum_g |V_g|} = \frac{\sum_g V_g \left[\frac{\sum_{j \in g} S_{ij} H_j Q_j}{\sum_{j \in g} H_j Q_j} \right]}{\sum_g |V_g|}.$$

A.9 Portfolio weights

$$w^* = \arg \max_w \left[\sum_i w_i \text{Score}_i - \lambda \sum_i w_i^2 \right]$$

subject to

$$\sum_i w_i = 1, \quad 0 \leq w_i \leq w_{\max}.$$

Defaults: $\lambda = 0.10$ and $w_{\max} = 0.30$.

A.10 Effective holdings

With $HHI = \sum_i w_i^2$:

$$N_{eff} = \frac{1}{\sum_i w_i^2}.$$

A.11 Rolling beta

$$\beta_w = \frac{\text{Cov}_w(r_s, r_m)}{\text{Var}_w(r_m)}.$$

A.12 Active return

$$r_{active,t} = r_{strategy,t} - r_{S\&P500,t}.$$

B Horizon-score configurations and broad portfolio sweep

The horizon-score catalogue contains 21 fine-grained Portfolio Target Types and 10 horizon keys, giving 210 possible cells. Each active cell uses its configured candidate range in 0.05 steps. The first search value is candidate index 2, insensitive cells can be frozen, random-context screens reduce the candidate lists, and the retained non-negative configurations are passed to broad simulation.

A retained configuration is stored as one complete mapping of target type and horizon to score. For readability, an example is shown in abbreviated form:

Portfolio Target Type	1m	1d	20d	252d
ALPHA	0.10	0.40	0.80	0.60
RELATIVE_ALPHA	0.05	0.35	0.90	0.70
CROSS_SECTION_ALPHA	0.10	0.45	0.95	0.50
DIRECTION	0.25	0.75	0.85	0.20
...				
TAIL_RISK	0.55	0.95	0.75	0.15
...				
EXECUTION	0.90	0.40	0.10	0.00
MARKET_IMPACT	0.95	0.30	0.05	0.00

The full stored configuration also includes the 5m, 15m, 60m, 5d, 60d and 120d keys and all omitted target types.

The broad portfolio sweep also uses:

Dimension	Values
Maximum weight	0.10; 0.15; 0.20; 0.25; 0.30
Concentration penalty	0.00; 0.05; 0.10; 0.20
Rebalance days	20; 40; 60; 90; 120
Fee parameter	0.00
Non-horizon configurations	up to 2,500 in the active setup
Horizon multiplier	one run per retained horizon-score configuration
Liquidity classification window	first 60 backtest trading days

The intended total is $2500 \times N_{\text{horizon configurations}}$, subject to runtime limits.

C Base universe definitions

C.1 High Liquidity 30

AAPL, MSFT, AMZN, NVDA, META, TSLA, AMD, INTC, QCOM, MU, CSCO, ORCL, JPM, BAC, WFC, C, XOM, CVX, F, GM, T, VZ, PFE, JNJ, WMT, DIS, GE, HD, NFLX, GOOG

C.2 Medium Liquidity 30

ADI, MCHP, TXN, STX, WDC, PNC, USB, BK, STT, COF, CAT, DE, EMR, ETN, ITW, LOW, TGT, KR, BBY, DRI, AMGN, GILD, BIIB, BMY, CVS, OXY, EOG, SLB, HAL, VLO

C.3 Lower Liquidity 30

AIT, ARCB, BRC, CALM, CHCO, CPK, CNMD, FFIN, GATX, GBCI, GFF, HNI, HVT, JJSF, LANC, MGEE, MLAB, MMSI, MTRN, NRIM, NWN, RCKY, RELL, SCL, TNC, UVSP, WASH, WDFC, NATH, RES

C.4 Sector-matched / Sector Spread 30

MSFT, ADI, RELL, JPM, PNC, CHCO, GE, CAT, AIT, AMZN, LOW, HVT, WMT, KR, JJSF, JNJ, AMGN, CNMD, XOM, OXY, RES, NEE, DUK, CPK, FCX, NUE, SCL, PLD, SPG, UHT

C.5 Liquidity Barbell 30

AAPL, MSFT, AMZN, NVDA, META, TSLA, JPM, BAC, XOM, CVX, WMT, JNJ, GE, HD, NFLX, AIT, ARCB, BRC, CALM, CHCO, CPK, CNMD, FFIN, GATX, HNI, JJSF, MGEE, MTRN, TNC, WDFC

The 60- and 90-stock definitions in `Data_Creation_Screening.py` are built from these base liquidity groups and keep their explicit ticker lists in code.

D Current validation/search constants

Setting	Value
Walk-forward validation window	20 dates
Model search random state	42
Minimum folds before performance pruning	3
Fold 3 racing stage	keep top 50%, minimum 5 per family
Fold 5 racing stage	keep top 50%, minimum 3 per family
Fold 7 racing stage	keep top 50%, minimum 2 per family
Mature finalists per target	up to 8
Partial legacy promotions per model family	up to 2
Finalist variability penalty	$0.25 \times \text{metric standard deviation}$
Continuous Rank IC stability gate	≤ 0.10
Binary ROC AUC stability gate	≤ 0.07
Multiclass Macro F1 stability gate	≤ 0.10
Horizon validation fraction	25% of research dates
Horizon score step	0.05
Horizon initial index	2
Horizon sensitivity freeze	$CV < 0.05$
Scheduled random horizon screens	20 @ 0.15; 50 @ 0.30; 100 @ 0.35
Preferred exhaustive-search trigger	$\leq 1,000$ remaining configurations
Manual override	allow exhaustive move-on above 1,000
Exhaustive retention	non-negative BQ, top 5%